METODOLOXÍAS DE DESENVOLVEMENTO

Agile

Anxo Pérez & Javier Parapar

anxo.pvila@udc.es

Information Retrieval Lab
Computer Science Department
University of A Coruña

Desarrollo Iterativo

Características del Desarrollo Iterativo

- El desarrollo iterativo es un enfoque de creación de software que divide el ciclo de vida en múltiples **iteraciones**, cada una de las cuales es un mini-proyecto completo.
- El objetivo es entregar una versión funcional del software al final de cada iteración, con un incremento constante en funcionalidad y calidad.
- A diferencia de los enfoques secuenciales, permite una adaptación continua a los cambios en los requisitos y retroalimentación del cliente.

Estructura de una Iteración

- Cada **iteración** se planifica como un mini-proyecto con un ciclo completo de desarrollo: requisitos, diseño, codificación y pruebas.
- El resultado de cada iteración es una versión parcial, estable e integrada del sistema, conocida como **release**.
- Las iteraciones posteriores construyen sobre las versiones anteriores, aumentando la funcionalidad y refinando la calidad del producto.
- A diferencia de un prototipo, cada release es un subconjunto real y operativo del sistema final.
- La release de la iteración final es un producto completo listo para el mercado o el cliente.

Desarrollo Iterativo e Incremental (IID)

- Normalmente, los sistemas crecen en funcionalidades entre iteración e iteración.
- Ampliar un sistema a través de iteraciones se conoce como **Desarrollo Iterativo e Incremental** (IID).
- Hoy en día, la mayoría de los métodos de desarrollo son IID, enfatizando la importancia de las iteraciones cortas y el feedback constante.
- IID está en el núcleo de las metodologías ágiles, incluyendo el Proceso Unificado, Scrum y XP.

¿Cuántas y cómo son las iteraciones?

- La mayoría de los proyectos incluyen al menos 3 iteraciones antes de una versión final, aunque no hay un límite máximo.
- Las iteraciones suelen durar entre 1 y 6 semanas, con una tendencia a utilizar ciclos más cortos para maximizar la retroalimentación y la adaptabilidad.
- Iteraciones más cortas permiten una evaluación más rápida y frecuente de los resultados, reduciendo el riesgo de desviaciones importantes en el proyecto.
- La duración de la iteración puede variar según la naturaleza del proyecto, pero es importante que sea lo suficientemente breve como para permitir la flexibilidad, sin ser tan corta que impida un progreso significativo.

Planificación Iterativa

Planificación Iterativa

¿Qué hacemos en las próximas dos/tres/cuatro semanas de iteración?

- La planificación de cada iteración es crítica para el éxito del proyecto.
- Se enfoca en determinar qué funcionalidades y tareas se abordarán en el próximo ciclo de trabajo.
- Es fundamental priorizar tareas de alto valor y riesgo para maximizar el impacto de cada iteración.

Definición de Prioridades en Iteraciones

- Los métodos de desarrollo iterativo incremental combinan dos enfoques para priorizar tareas:
  - **Basado en Riesgo:** Enfoque en tareas de mayor riesgo en las primeras iteraciones para minimizar problemas futuros.
  - **Basado en el Cliente:** Priorización según el valor de negocio percibido por el cliente.
- Se recomienda aplicar ambos esquemas, ya que el cliente no siempre aprecia lo que es técnicamente complejo, y los desarrolladores no siempre aprecian lo que tiene alto valor de negocio.

Basado en riesgo técnico

Ejemplo: una tienda en línea

- **Prioridad:** Validar la integración con la pasarela de pago, que el equipo nunca ha utilizado.
- **Riesgo:** Un fallo al confirmar un pago podría generar cobros duplicados o pedidos sin registrar.
- **Primera iteración:** Implementar una compra mínima en el entorno de pruebas y comprobar pagos aceptados, rechazados y reintentos.
- Resolver esta incertidumbre pronto permite ajustar el diseño antes de construir el resto del proceso de compra.

La misma tienda en línea

- **Prioridad:** Permitir consultar productos y completar una compra sencilla.
- **Valor:** La empresa puede empezar a vender y comprobar si el proceso de compra satisface a sus usuarios.
- **Primera versión:** Un catálogo reducido con compra y confirmación del pedido. Las recomendaciones y los cupones pueden esperar.
- **Ambos criterios se combinan:** La compra aporta valor y permite validar la integración de pago de mayor riesgo.

Desarrollo Iterativo con Timeboxing

Timeboxing en Iteraciones

- **Timeboxing** es la práctica de fijar la duración de cada iteración, sin permitir que se modifique la fecha de finalización (Como suele suceder con la fecha de finalización del proyecto).
- Si se identifica que no se cumplirán todas las tareas planificadas, se ajusta el alcance, priorizando lo más importante.
- El objetivo es entregar una versión estable y probada al final de cada iteración, incluso si se reduce el alcance de las funcionalidades.
- Scrum y XP son metodologías que recomiendan encarecidamente el uso de timeboxing para mantener un ritmo de trabajo sostenible y predecible.

Gestión de la Carga de Trabajo con Timeboxing

- La limitación del timeboxing no debe llevar a aumentar la presión sobre los desarrolladores.
- La longitud de las iteraciones no tiene por qué tener la misma duración.
- Si se detecta que no se alcanzará el objetivo, se debe reducir el alcance de la iteración en lugar de aumentar la carga de trabajo.
- La clave es mantener un ritmo sostenible que garantice la calidad y estabilidad del producto en cada release.

Cambios en la iteración

- Una vez que las solicitudes para una iteración han sido definidas, bajo ninguna circunstancia un implicado externo puede cambiar el trabajo.
- Las metodologías ágiles e iterativas abrazan el cambio, pero no el caos.

Desarrollo Evolutivo - Adaptativo

Desarrollo Evolutivo y Adaptativo

- El desarrollo en iteraciones propuesto por las metodologías ágiles se basa en seguir un enfoque **adaptativo** y **evolutivo**.
- A diferencia del enfoque secuencial, se asume que los requisitos no pueden ser completamente definidos al inicio del proyecto.
- El desarrollo **adaptativo** enfatiza la importancia del feedback continuo de usuarios, testers y desarrolladores para ajustar el rumbo del proyecto.
- Este enfoque es fundamental para proyectos en entornos dinámicos, donde los requisitos pueden cambiar rápidamente en respuesta a nuevas necesidades o condiciones del mercado.

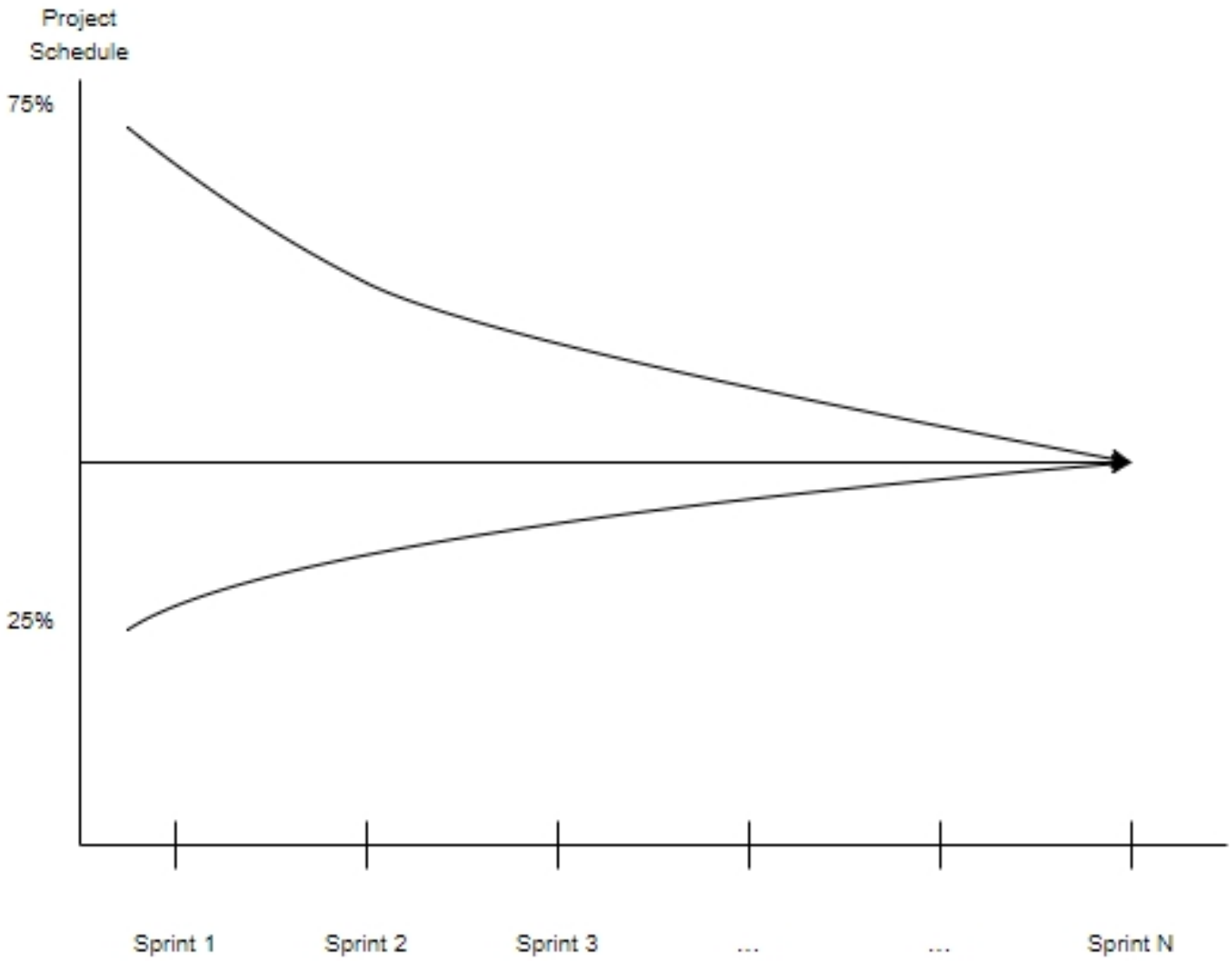
Análisis de requisitos en agile

- No es cierto que no se puedan definir unos requisitos definidos o que siempre estén cambiando.
  - La mayoría de los requisitos son definidos en las primeras iteraciones. Ciertas metodologías iterativas recomiendan talleres de requisitos de uno o dos días en cada iteración.
- No es cierto que sea necesario conocer el 100% de los requisitos funcionales para comenzar a construir un sistema.
  - La importancia de identificar el *Top Ten* de los requisitos de alto nivel.

Planificación

- Tampoco es cierto que no exista planificación o que esta sea casi desconocida.
- Existe el concepto del *Cono de incertidumbre*:
  - Se estima desde el inicio mediante rangos, que se revisan con la evidencia obtenida en las primeras iteraciones antes de asumir compromisos más precisos.
  - Se recomienda la planificación adaptativa sobre la planificación predictiva.
  - **Problema?:** Si entramos en contratos fijos, debido a la falta de flexibilidad para gestionar cambios en el alcance, costos y cronogramas, algo que en metodologías ágiles se considera parte del proceso natural.

Cono de incertidumbre



El cono de incertidumbre en Scrum

- **Al inicio:** Sabemos menos sobre los requisitos, las dificultades técnicas y la capacidad del equipo. Por eso, la estimación del plazo tiene un margen amplio.
- **Con cada sprint:** El trabajo terminado y la revisión con el cliente aportan información para ajustar el alcance y actualizar las previsiones.
- **El cono se estrecha:** Al resolver dudas, podemos estimar con mayor precisión cuándo estará listo el producto. La gráfica es ilustrativa, no garantiza unos porcentajes de precisión.
- **No ocurre automáticamente:** La incertidumbre disminuye gracias al aprendizaje. Nuevos requisitos o riesgos pueden volver a ampliar el rango.

Planificación y Entrega en Desarrollo Adaptativo

- El *Cono de Incertidumbre* representa cómo puede reducirse la incertidumbre de las estimaciones al aclarar el alcance y resolver riesgos durante el desarrollo.
- La planificación adaptativa, que ajusta el plan a medida que se obtiene más información, es preferible a la planificación predictiva, especialmente en entornos cambiantes.

Desarrollo Evolutivo - Adaptativo

- La entrega incremental supone la puesta en producción de versiones en el tiempo incrementando sus funcionalidades. No se debe confundir *entrega incremental* con desarrollo iterativo.
  - **Entrega Incremental:** Se refiere a la puesta en producción de versiones incrementales del sistema, con funcionalidades adicionales en cada versión.
  - **Entrega Evolutiva-Adaptativa:** Resalta la importancia de usar el feedback de las versiones en producción para definir las metas de las versiones futuras.

Desarrollo Ágil

Conceptos

- **Agilidad:** Capacidad de responder rápida y flexiblemente al cambio en cualquier etapa del desarrollo.
- El lema de los métodos ágiles es *"abrazar el cambio"* (*embrace change*), aceptando que el cambio es inevitable y puede ser una ventaja competitiva.

Desarrollo Ágil

Los métodos de **desarrollo ágil** aplican:

- desarrollo **iterativo**
- **iteraciones de duración fija** (*timeboxed*)
- **planificación adaptativa**
- promueven las entregas evolutivas
- incluyen otros valores y prácticas que fomentan la agilidad.

Elementos Clave

- **Desarrollo iterativo:** Se basa en la creación de incrementos del producto a través de iteraciones cortas y repetitivas.
- **Iteraciones de duración fija (timeboxing):** Cada iteración tiene una duración fija, generalmente entre 1 y 4 semanas.
- **Planificación adaptativa:** El plan del proyecto se ajusta continuamente basado en la retroalimentación y los cambios en los requisitos.
- **Entregas evolutivas:** Se realiza una entrega continua y gradual del producto, con valor agregado en cada iteración.
- Otros valores y prácticas fomentan la agilidad, como la colaboración, la mejora continua y la simplicidad.

Algunas Prácticas

Scrum

- **Sala de proyectos compartida:** Espacio donde todo el equipo trabaja junto, facilitando la comunicación y la colaboración.
- **Equipos auto-organizados:** Equipos que gestionan su propio trabajo sin una dirección centralizada.
- **Reuniones diarias de pie (*Scrum meetings*):** Reuniones cortas para sincronizar el trabajo del equipo y resolver problemas rápidamente.

eXtreme Programming (XP)

- **Tarjetas de historia:** Uso de tarjetas para resumir los requisitos de manera sencilla y accesible.
- ***Pair programming*:** Dos desarrolladores trabajan juntos en una misma tarea, mejorando la calidad del código.
- **Participación activa del cliente:** El cliente está involucrado de manera continua, proporcionando retroalimentación directa.

Manifiesto Ágil

Manifiesto Ágil

- **Publicado en 2001** por 17 desarrolladores de software en respuesta a los enfoques rígidos y burocráticos en los proyectos de desarrollo.
- Define una serie de valores y principios que buscan optimizar la manera en que se desarrollan los proyectos de software, promoviendo la colaboración, la flexibilidad y el valor entregado al cliente.

Manifiesto Ágil

Estamos descubriendo mejores formas de desarrollar
software haciéndolo y ayudando a otros a hacerlo.
A través de este trabajo hemos llegado a valorar:

Individuos e interacciones sobre procesos y herramientas
Software funcionando sobre documentación exhaustiva
Colaboración con el cliente sobre negociación contractual
Responder al cambio sobre seguir un plan

Es decir, aunque hay valor en los elementos de la derecha,
valoramos más los de la izquierda.

Fuente: http://agilemanifesto.org/ ©Los autores mencionados

Autores del Manifiesto Ágil

Los autores originales del Manifiesto Ágil incluyen:

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler
James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick
Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Más información disponible en:
`http://agilemanifesto.org/authors.html`

Los Doce Principios del Manifiesto Ágil

Seguimos estos principios:

1. Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor.
2. Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.
3. Entregamos software funcional frecuentemente, entre dos semanas y dos meses, con preferencia al período de tiempo más corto posible.
4. Los responsables de negocio y los desarrolladores trabajamos juntos a diario durante todo el proyecto.

Los Doce Principios del Manifiesto Ágil

5. Los proyectos se desarrollan en torno a individuos motivados. Es necesario darles un entorno y el apoyo que necesitan, y confiarles la ejecución del trabajo.
6. El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es una conversación cara a cara.
7. El software funcionando es la medida principal de progreso.
8. Los procesos ágiles promueven el desarrollo sostenible. Los promotores, desarrolladores y usuarios deben ser capaces de mantener un ritmo constante de forma indefinida.

Los Doce Principios del Manifiesto Ágil

9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.
10. La simplicidad, elegir la opción que aporte más valor empleando el menor esfuerzo posible, es esencial.
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos auto-organizados.
12. A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo para luego ajustar y perfeccionar su comportamiento en consecuencia.